UNIVERSITI MALAYA

FAKULTI SAINS KOMPUTER DAN TEKNOLOGI MAKLUMAT
Faculty of Computer Science and Information Technology

WOC7017 BIG DATA PROCESSING SEMESTER 2, 2023/2024

SECTION 1

PROJECT TITLE:

Flipkart Sales Analysis Using Spark & Hadoop

PREPARED BY:

NAME	MATRIC NO.
SYED MASHKUR BILLAH	22091926

SUPERVISED BY:

PROFESOR MADYA DR. MUMTAZ BEGUM BINTI PEER MUSTAFA

Table of Contents

1.0 Introduction to Domain 3
1.1 Project Objective 3
1.2 Expected Outcome 4
1.3 Dataset 4
1.4 Tools Used 5

2.0 Literature Review 5
2.1 Hadoop in Sales and E-Commerce 5
2.2 Spark in Sales and E-Commerce 6

3.0 Experimental Setup 7
3.1 Dataset Description 7
3.2 Data Pre-Processing 9
3.2.1 Data Cleaning 9
3.2.2 Data Merging 11
3.2.3 Data Validation 12

4.0 Case Studies 16
4.1 Case Study 01 16
4.2 Case Study 02 21
4.3 Case Study 03 27
4.4 Case Study 04 33

5.0 Conclusion 40

1.0 Introduction to Domain

In this case study, we will use the e-commerce area for the purpose of focusing of this big data analysis. Big data has turned into a real treasure for businesses looking to capitalize on consumer behavior and refine their sales associating tactics. Flipkart, which is the largest e-commerce marketplace in India by market share, produces enormous transactional and product data on a daily basis. Started in 2007, it is one of the biggest players in the Indian E-Commerce market. Amazon started as an online bookstore and scaled to a wide variety of product categories as diverse as consumer electronics, fashion, home essentials, groceries, and lifestyle products. Founded in 2007, Flipkart has played a pivotal role in shaping India's e-commerce ecosystem and growing the nation's burgeoning e-commerce market. This report aims to delve into a comprehensive analysis of Flipkart's sales data, encompassing a dataset of approximately 5 GB that encapsulates all sales and product information.

This analysis is basically targeted to actionable insights that will help in strategic decision making process. We will dig into the dataset by going through proper preparation and inspection and find insights - whether it be consumer areas, sales trends and many more regarding the type of product range and peak sales that emerge during the seasons. These case studies will also present within the data, which will contextualize our data and give my findings depth.

During this analytical journey, we will be using some datagician ways of processing the vast amount of data. The methodology will be clear and ensure that the dataset collected is clean, legible and good enough to make the kind of analysis required. I intend to extract insights into how customers behave on Flipkart so as to offer actionable insights that might improve the platform for better performance and customer satisfaction.

Numbers in this big data sales analysis using Spark and Hadoop are not just that, numbers, they have some narrative way behind them. This is really about understanding the intricacies of the psychology behind decisions about purchase you are making and how to tap into that to make shopping even more engaging and personal. I hope this report adds to the work Flipkart is doing to stay ahead of the game in the e-Commerce space.

1.1 Project Objective

The objective of this project with the Flipkart Sales Dataset is focused to extract actionable business insights that can inform strategic decisions. Here’s a structured way to define the objective:

- To Analyze the Purchase Patterns – Enhance consumer purchase behavior Identify the most popular e-commerce brand or products

- To Identify the best-selling products and maintain inventory levels to avoid stocking and out of stock.

- To Enable identification of segments of customers and targeted marketing campaigns driving more engagement and sales.

- To assess client vs. competitor popularity & brand discounts

- To examin city-wise order distribution to improve and optimize the logistics and delivery process

- To examine the effect of discounts on sales volumes and customer acquisition in various cities.

The end goal is using data modeling to drive growth and customer satisfaction, reduce cost of operations, and overall improve profitability at Flipkart. Every case study that will be executed, will be in service towards this goal by solving different problem in the E commerce ecosystem.

1.2 Expected Outcome

In addition I will suggest proactive measures for Flipkart to do better for its customers, become more efficient operationally, and achieve higher profits. It is designed as a decision-support for stakeholders, laying a data-driven foundation for the following business steps. The analysis will not only depict the current scenario but will also lead the direction for predictive & prescriptive analytics coming in picture - laying the foundation for a greater proactive business moves into one of the most rapidly changing e-commerce industry.

1.3 Dataset

The dataset is a 4.57 GB dataset and has a total of 13 columns/variables and has more than million rows (1,048,576 rows), but given the fact that I will reduce by selecting some more relevant columns and deleting others to reduce the size issue. The collection of this sales data takes place from April to July of 2022. There are some ambiguous information with the data contents i.e. unnecessary variables, blank values, incorrect data which are not necessarily needed for this specific data analysis and also the column names are misleading that would need some cleanings as well. The dataset is taken from an open source website called Kaggle and it can be found here at: https://www.kaggle.com/datasets/iyumrahul/flipkartsalesdataset

1.4 Tools Used

The tools, packages and the frameworks that I have used to accomplish this large sales data analysis are Hadoop and Spark. Both Hadoop and Spark have been used to compare the analysis results for the same case studies. For Hadoop framework I have used python as my core programming language to run code using a virtual environment in Ubuntu operating system. On the other hand, for Spark framework, I have used jupyter notebook to execute the codes written in python programming language as well. Moreover, HDFS, Hadoop Streaming, pySpark are some of the important modules and packages used for execution of the python programs for my case studies.

2.0 Literature Review

Big data has revolutionized many industries, especially those of sales and e-commerce platforms that have benefited largely from that. The main thing that has brought this transformation is Apache Spark and Apache Hadoop which are well known open source data processing frameworks. This Literature Review would help one to find the surfaced results through popular sales domains and e-commerce space in context of Hadoop and Spark.

2.1 Hadoop in Sales and E-Commerce

E-commerce Platforms rely on Hadoop, an open-source framework for storing and processing to keep up with the enormous amount of data they produce. In an e-commerce application with immense data volumes, it becomes necessary to store and process large amounts of unstructured and structured data, thereby making Hadoop the ideal framework for e-commerce applications is one of the best possible choices (White, 2012). HDFS, the distributed file system of the framework also supports the cost effective scaling of data storage essential for rapidly growing e-commerce platforms (Ghemawat, Gobioff, & Leung, 2003).

Hadoop for E-Commerce: It is widely used for building e-commerce platforms, where it supports applications like customer behavior analysis, recommendation system, sentiment analysis, etc. For example, (Chen, Mislove, & Wilson, 2015) showed that Hadoop enables a company to process millions of customer reviews and social media messages and analyze the spectrum of opinion to derive insights into customer preferences and satisfaction. Also, the Hadoop ecosystem, with tools like Hive and Pig, provides the ability to run queries and process raw data into meaningful insights, which improves decision-making (Thusoo et al., 2010).

2.2 Spark in Sales and E-Commerce

E-commerce platforms and Apache Spark With its speed and advanced analytics, Apache Spark has just taken e-commerce platforms to a much different level. The in-memory computing framework of Spark accelerates processing to enable real-time analytics (Zaharia et al., 2012). This is especially critical for e-commerce companies that need real-time data-driven decisions to maximize efficiencies, deliver more personalized customer experiences, and improve their marketing strategy.

As Spark works with e-commerce business with machine learning libraries like MLlib, in the same way, they can employ a more advanced predictive model. As highlighted by Meng et al. (2016) the enterprise's new recommendation engine, developed with Spark's machine learning tools, would analyze user behavior to suggest products to them in real-time and drive growth in sales and customer interaction. Additionally, Spark Streaming (which provides an efficient way processing streaming data) is pretty important to be able to observe and analyze real-time customer interactions and transactions (Zaharia et al., 2013).

The use of Hadoop, combined with Spark in particular, has been shown to enable a substantial reduction in operational costs and customer satisfaction in a variety of studies. For example, Wu, Zhu, Wu, and Ding (2014) described how an e-commerce platform successfully integrated Hadoop and Spark, which cut the data processing time in half, helping the recommendation system be both quick and accurate. The integration not only enhances the performance of the platform but also provides customers with a personalized experience by offering recommendations, as well as timely responses to customer inquiries.

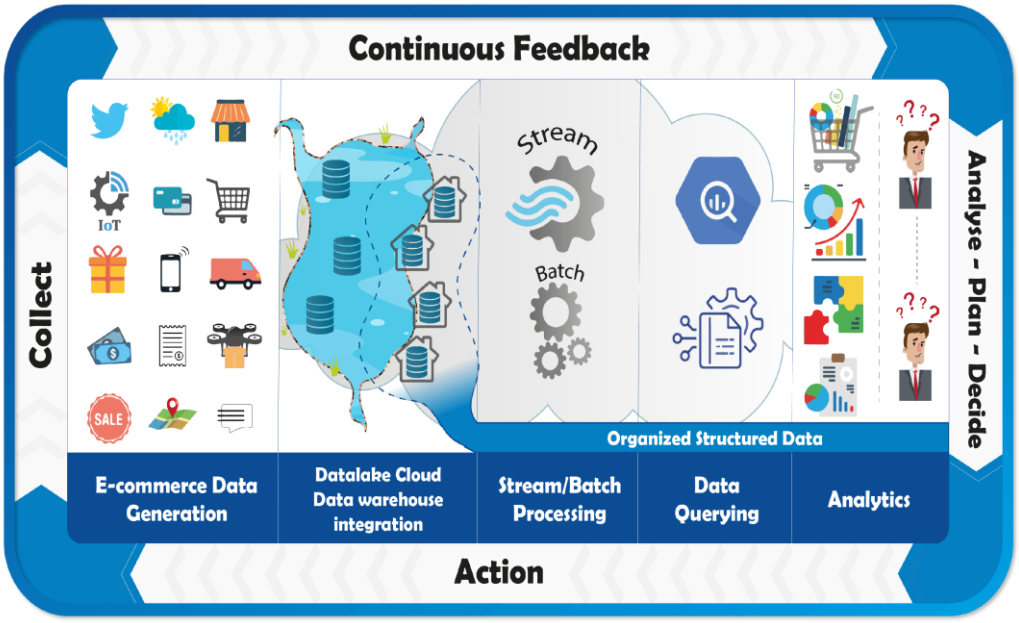


Fig 1: Data Processing Flow in E-Commerce using Hadoop and Spark

3.0 Experimental Setup

3.1 Dataset Description

The dataset consists of 2 csv formatted files called sales.csv and products.csv respectively. This dataset basically holds the details of the sales of specific orders in different cities listed. The sales dataset file has 13 columns and has a total rows of 1,048,576. It holds the size of 4.57 GB alone. This data contains various attributes related to sales transactions, such as product details, pricing, sales volumes, and potentially customer information. The abstract of the dataset is shown below in a table to give a short idea:

Sales.csv		
Variable name	**Defined as**	**Remarks**
#	Blank ID	It is just a blank column with no values and is totally unnecessary
Unnamed: 0.1	This is a random ID renegerator	This is also an unused column and has no meaning. It will also be omitted onwards before doing analysis.
Unnamed: 0	This is another random ID regenerator	This is also an unused column and has no meaning. It will also be omitted onwards before doing analysis.
Date_	Date column	This column represents the date of purchase of an item or order by the consumer
City_name	Name of the city	This is the name of the city where the order is created from
Order_id	Holds the order id	This is the field where the order_id is generated for a particular item that was purchased
Cart_id	Id of the cart	This is the ID of the cart that was used by the consumer to purchase a product or multiple products
Dim_customer_key	Customer ID	This is a very useful column stating the specific key id of a customer who ordered an item
Procured_quantity	Quantity of the order	This is the field where the quantity of the items is stored
Unit_selling_price	Per unit price	This is thed selling price for a unit
Total_discount_amount	The discount for an order	This column stores the total discount given to a specific order to a customer
Product_id	Id of the product	This column holds the specific ID of a specific product that was ordered

Total_weighted_landing_price	Total price	This is the column where it stores the total price after giving the discount which is already paid by the consumer

The second dataset is called products.csv file which consists 13 columns as well and has a total rows of 32227. This dataset basically holds the details of the products listed. It holds the size of around 6 MB alone. Later after merging the dataset into one, the size will increase up to 5 GB as the final dataset after some data cleaning and validations.

The abstract of the dataset is shown below in a table to give a short idea:

Products.csv		
Variable name	**Defined as**	**Remarks**
#	Blank ID	It is just a blank column with no values and is totally unnecessary
Product_id	Id of the product	This column holds the specific ID of a specific product that was ordered
Product_name	Name of the product	This is the column where the product name is stored to order
Unit	Unit of the product	This column stores the specific amount or unit of the product i.e 1 unit or 500g etc.
Product_type	Type of the product	This variable stores the type of the product where it falls
Brand_name	Name of the brand	This will show the name of the brands under which the products are being manufactured
Manufacturer_name	Name of the manufacturer	This column stores the product manufacturer name
I0_category	The category of product	This is the column where the category of the product is stored
I1_category	The category of product	This is the column where the category of the product is stored
I2_category	The overall category of products	This is the column where the overall category of the product is stored. Note that this column will give the overall category of the product.
I0_category_id	The category ID	This column will store the category IDs for the specified categories in I0_category.
I1_category_id	The category ID	This column will store the category IDs for the specified categories in I1_category.
I2_category_id	The category ID	This column will store the category IDs for the specified categories in I2_category.

3.2 Data Pre-Processing

In this phase, I proceeded with the process of taking raw data and converting it into usable information. In order to do this, I moved forward taking each point and note it in a step-by-step approach. As the both of the sales.csv and products.csv files are not pre-processed well, I tried to gather, filter, sort, process, analyses, and store them before being displayed in a usable way. These are the steps that I followed while processing with the data:

3.2.1 Data Cleaning

After reviewing the data, I have discovered that there are some unnecessary data involved in the datasets. I have used python programming language and pySpark framework in order to prepare the datasets before moving forward to the analysis part of the case studies.

Sales Data:

1. First of all, in the sales.csv dataset, I started data cleaning with removing the unnamed columns with meaningless IDs. These columns involve *'_c0', 'Unnamed: 0.1', 'Unnamed: 0', 'cart_id'*. This actually reduced the row count of the whole dataset by 79,355.

```
[3]: # To remove multiple columns by name
     df = df.drop('_c0', 'Unnamed: 0.1', 'Unnamed: 0', 'cart_id')

     # Show the updated DataFrame schema
     df.printSchema()
     # Display the first few rows of the DataFrame
     df.show(10)

     root
      |-- date_: date (nullable = true)
      |-- city_name: string (nullable = true)
      |-- order_id: integer (nullable = true)
      |-- dim_customer_key: integer (nullable = true)
      |-- procured_quantity: integer (nullable = true)
      |-- unit_selling_price: double (nullable = true)
      |-- total_discount_amount: double (nullable = true)
      |-- product_id: integer (nullable = true)
      |-- total_weighted_landing_price: double (nullable = true)
```

2. Secondly, to remove any duplicate rows, I ran the *.dropDulicates()* function in pySpark to remove the duplicate rows based on all the columns.
3. To drop rows with any null values in order to handle missing values I have used the na functions to deal with nulls or missing values in various ways.

4. After renaming columns and dealing with some unusual variables and handling missing values, I have also changed some of the column names in this dataset to identify the proper use of that variable/column, such as :

Column Name	Changed to
date_	purchase_date
procured_quantity	quantity

Now, after processing this dataset, I have found that the final row count of this dataset is 46627032 rows in total:

```
[6]: #after cleaning
     df.count()

[6]: 46627032
```

Products Data:

1. For this case, in the products.csv dataset, I started data cleaning with removing the unnamed columns with meaningless IDs as well as some category_id related fields which in my case is not relevant based on the case studies that I will perform. These columns involve *'_c0', 'unit', 'l1_category', 'l2_category', 'l0_category_id', 'l1_category_id', 'l2_category_id'*.

```
[12]: # To remove multiple columns by name
      df2 = df2.drop('_c0', 'unit', 'l1_category', 'l2_category', 'l0_category_id', 'l1_category_id', 'l2_category_id')

      # Show the updated DataFrame schema
      df2.printSchema()
      # Display the first few rows of the DataFrame
      df2.show(10)

      root
       |-- product_id: integer (nullable = true)
       |-- product_name: string (nullable = true)
       |-- product_type: string (nullable = true)
       |-- brand_name: string (nullable = true)
       |-- manufacturer_name: string (nullable = true)
       |-- l0_category: string (nullable = true)
```

2. Secondly, to remove any duplicate rows, I ran the *.dropDulicates()* function in pySpark to remove the duplicate rows based on all the columns.
3. To drop rows with any null values in order to handle missing values I have used the na functions to deal with nulls or missing values in various ways. This actually reduced the row count of the whole dataset by 3196.

4. After renaming columns and dealing with some unusual variables and handling missing values, I have also changed some of the column names in this dataset to identify the proper use of that variable/column, such as :

Column Name	Changed to
l0_category	product_category

Now, after processing this dataset, I have found that the final row count of this dataset is 29030 rows in total:

```
[17]: #after cleaning
      df2.count()

[17]: 29030
```

3.2.2 Data Merging

After, the data cleaning step is performed on both of the sales and products dataset, I have merged the 2 datasets into one by performing the joining operation. To join the 2 datasets I have used the *'product_id'* column as foreign key based on which the datasets will be joined. As this is the primary key to the products and is present in both of the datasets, I have taken this as reference. The final count of the dataset gives me 34143216.

```
[20]: # Joining df1 and df2 DataFrames and 'product_id' is the common column
      # Join the DataFrames using the 'product_id' column as a list
      joined_df = df.join(df2, ['product_id'])

[21]: joined_df.count()

[21]: 34143216
```

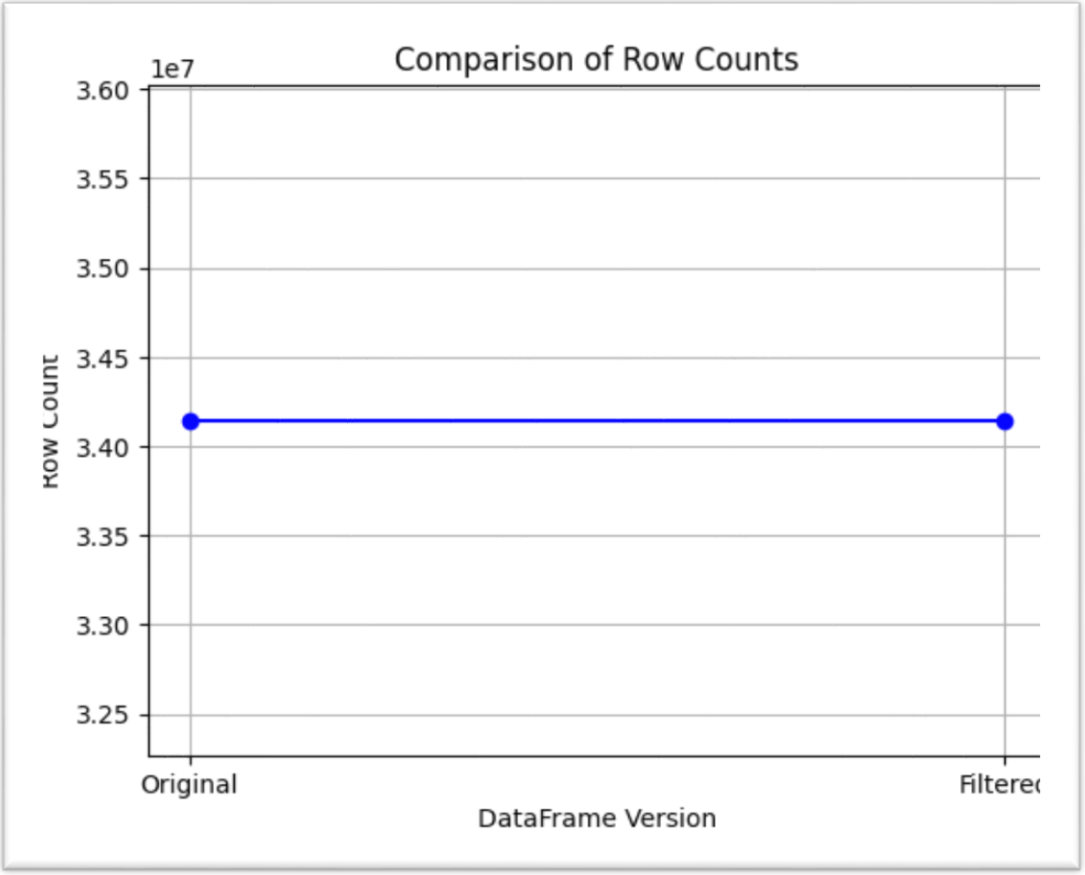
Now, this is what the final dataset schema looks like after data cleaning and merging:

```
[22]: # Show the updated DataFrame schema
      joined_df.printSchema()
      # Display the first few rows of the DataFrame
      joined_df.show(10)

      root
       |-- product_id: integer (nullable = true)
       |-- purchase_date: date (nullable = true)
       |-- city_name: string (nullable = true)
       |-- order_id: integer (nullable = true)
       |-- dim_customer_key: integer (nullable = true)
       |-- quantity: integer (nullable = true)
       |-- unit_selling_price: double (nullable = true)
       |-- total_discount_amount: double (nullable = true)
       |-- total_weighted_landing_price: double (nullable = true)
       |-- product_name: string (nullable = true)
       |-- product_type: string (nullable = true)
       |-- brand_name: string (nullable = true)
       |-- manufacturer_name: string (nullable = true)
       |-- product_category: string (nullable = true)
```

3.2.3 Data Validation

- **Date format check:** To validate if the purchase dates of the products are in good shape and are valid, I performed a cross check date validation to check the formatting of dates. For this, I have addeda new column *'purchase_date_valid'* which flags invalid date formats as null in the dataset. And finally used the *data.isNotNull().count()* to identify and filter out records with invalid dates and store them into a new filtered dataset for comparison with the original dataset. After counting both filtered and original datasets, I have found that both have the same row counts which means that there are no invalid date formatting in the original dataset:

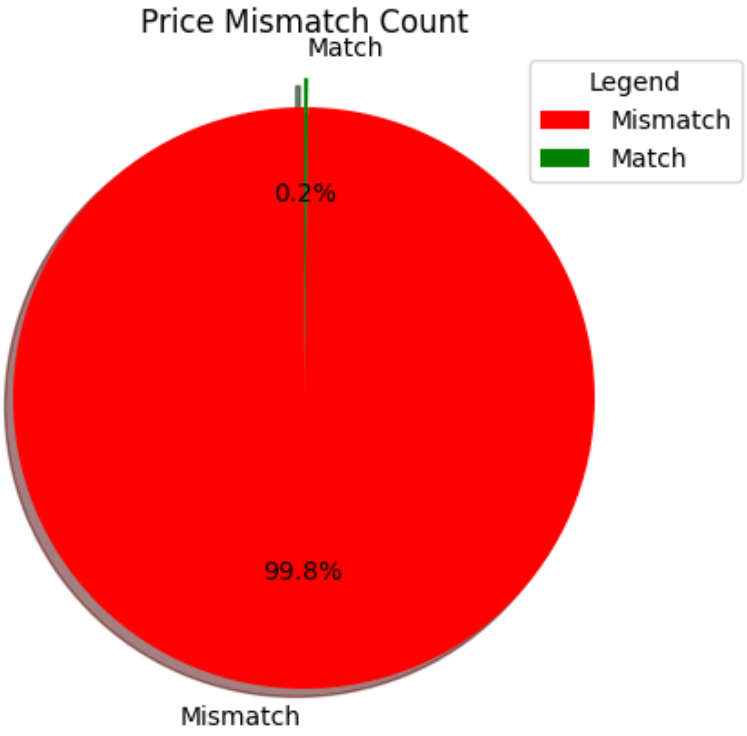


- **Calculation Check for total price:** In this case scenario, I checked with the *total_weighted_landing_price* column values if it correctly calculates the total price of an ordered item based on the unit price, quantity and the total discount given. As following the formula, which is *Total_weighted_landing_price = Quantity*Unit_selling_price – total_discount*, I have created a filtered dataset with additional columns for calculating the total price based on the identifiers given in the dataset and also put a column called *'price_mismatch'* as a boolean field to provide true/false values based on the comparison between the total price given and my calculated total price that I got by using the total price formula calculation. If my calculated result for the total price matches with the total price given in the dataset then the '*price_mismatch'* column will store false value otherwise it will store true value. Based on this criteria, I can easily point out the wrong calculated total weighted price for any order in the dataset and correct them by replacing with the right/correct value.

```
[26]: # Calculation Check for total_price
from pyspark.sql.functions import when
filtered_dataset = filtered_dataset.withColumn('calculated_total_price', col('quantity') * col('unit_selling_price') - col('total_discount_amount'))
filtered_dataset = filtered_dataset.withColumn('price_mismatch', when(col('total_weighted_landing_price') != col('calculated_total_price'), True).otherwi

filtered_dataset.show(8)

+----------+-------------+---------+---------+----------------+--------+-----------------+--------------------+---------------------------+----------
---------+--------------------+----------+--------------------+--------------------+-----------------+---------------------+-------------+
|product_id|purchase_date|city_name| order_id|dim_customer_key|quantity|unit_selling_price|total_discount_amount|total_weighted_landing_price|        pro
duct_name|        product_type|brand_name|   manufacturer_name|    product_category|purchase_date_valid|calculated_total_price|price_mismatch|
+----------+-------------+---------+---------+----------------+--------+-----------------+--------------------+---------------------------+----------
---------+--------------------+----------+--------------------+--------------------+-----------------+---------------------+-------------+
|    443262|   2022-04-01|   HR-NCR|112247955|        14725039|       1|            249.0|                 0.0|                   206.70306|Nivea Pure
Gentle...|                Talc|     Nivea| Nivea India Pvt Ltd|       Personal Care|         2022-04-01|                 249.0|         true|
|    368445|   2022-04-01|   HR-NCR|112266922|         5172327|       1|            100.0|                 0.0|                     86.964|Kissan Fres
h Toma...|      Tomato Ketchup|    Kissan|Hindustan Unileve...|    Sauces & Spreads|         2022-04-01|                 100.0|         true|
|     11186|   2022-04-01|    Delhi|112307176|        11288483|       1|             44.0|                 0.0|                     35.102|Catch Cumin
Powde...|Cumin Powder / Je...|     Catch|  D S Group (Spices)|Dry Fruits, Masal...|         2022-04-01|                  44.0|         true|
|    439935|   2022-04-01|Bengaluru|112306752|        17585641|       1|            115.0|                 0.0|                   103.05988|Grocery Arh
ar Dal...|           Arhar Dal|   Grocery|                HOT|    Atta, Rice & Dal|         2022-04-01|                 115.0|         true|
|     18851|   2022-04-01|    Delhi|112368359|         3788452|       1|             52.0|                 0.0|                     42.82462|Catch Coria
nder P...|    Coriander Powder|     Catch|  D S Group (Spices)|Dry Fruits, Masal...|         2022-04-01|                  52.0|         true|
```

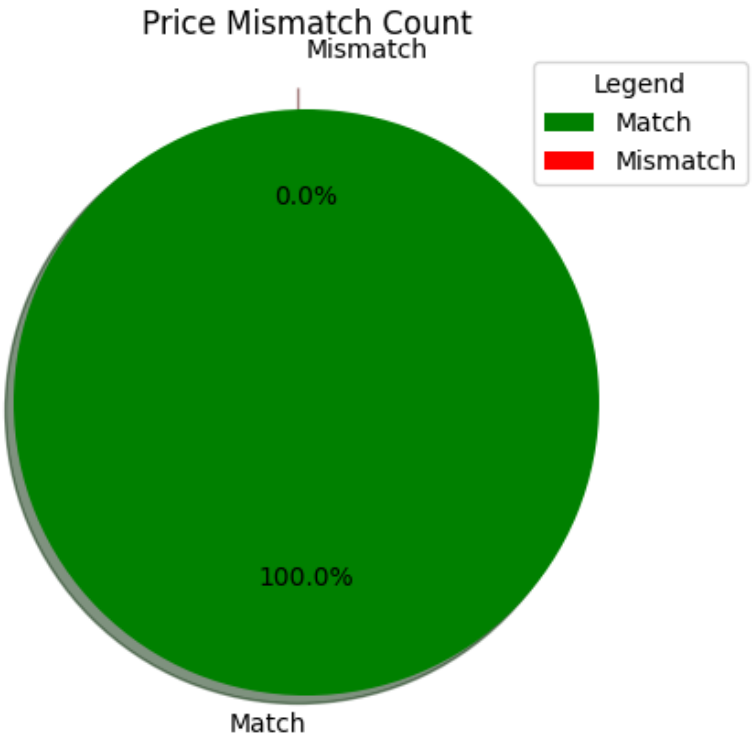


- As we can see that most of the values are incorrect for the total price calculation. There has been only 0.2% of the whole dataset are calculated correctly and the rest of the 99.8% data is calculated wrong. To resolve this issue, I have filtered out the wrong values with correct values for total weighted price calculation and re checked with the validation again:

```
[28]: # Filter out the wrong values
corrected_filtered_dataset = filtered_dataset.withColumn('total_weighted_landing_price',
                              when(col('price_mismatch'), col('calculated_total_price'))
                              .otherwise(col('total_weighted_landing_price')))


corrected_filtered_dataset.show(6)

+----------+-------------+---------+---------+----------------+--------+-----------------+--------------------+----------
---------+--------------------+----------+--------------------+--------------------+-----------------+---------------------
|product_id|purchase_date|city_name| order_id|dim_customer_key|quantity|unit_selling_price|total_discount_amount|total_weig
duct_name|        product_type|brand_name|   manufacturer_name|    product_category|purchase_date_valid|calculated_total_pr
+----------+-------------+---------+---------+----------------+--------+-----------------+--------------------+----------
---------+--------------------+----------+--------------------+--------------------+-----------------+---------------------
|    443262|   2022-04-01|   HR-NCR|112247955|        14725039|       1|            249.0|                 0.0|
Gentle...|                Talc|     Nivea| Nivea India Pvt Ltd|       Personal Care|         2022-04-01|                 24
|    368445|   2022-04-01|   HR-NCR|112266922|         5172327|       1|            100.0|                 0.0|
h Toma...|      Tomato Ketchup|    Kissan|Hindustan Unileve...|    Sauces & Spreads|         2022-04-01|                 10
|     11186|   2022-04-01|    Delhi|112307176|        11288483|       1|             44.0|                 0.0|
```



Now, it can be seen that all the results are corrected and there are no price mismatch values which concludes that all the total price calculate are correct.

3.2.4 Data Saving and Correlation Analysis

After finishing the data validation, the final dataset is saved as a new csv formatted file as *'cleaned_dataset.csv'* which I will use onwards for analyzing my case studies. The correlation matrix of the final dataset is shown below:

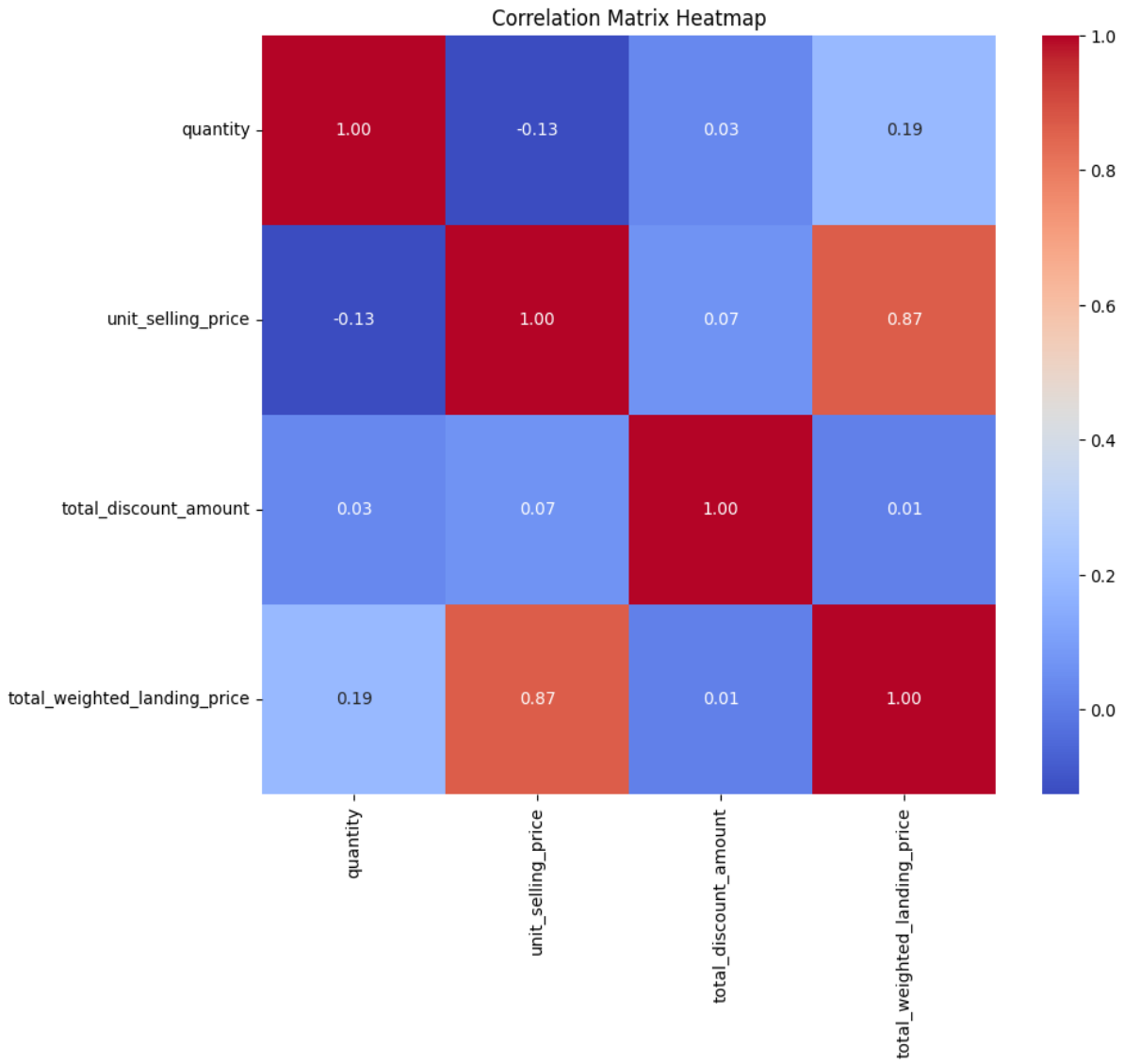


Fig 2: Correlation Matrix for the final dataset

As we can notice from the final dataset correlation heat map, the total weighted landing price totally depends on the unit selling price and it's proportional to each other. Quantity of the items does also have a mild relation with the total weighted landing price of an order. However, there were no relation with the total discount and the quantity of the product orders which means that based on this data the customers show not much of an interest towards the discounts given, rather they buy the products they need anyway. This means the customers were not truly dependent on how much discounts are given to an order or they have been offered.

4.0 Case Studies

4.1 Case Study 01:

- **Which manufacturer was the most popular in terms of getting orders than others? (Ranking based on items ordered).**

Key-Value Pair:

Key	Value
Manufacturer	1 (each order is counted as 1)

In this MapReduce job, the mapping phase emits a pair for each order with the Manufacturer as the key and a count of 1. The reduce phase then sums these counts to get the total number of orders for each manufacturer, allowing us to rank the manufacturers by their popularity based on the number of orders.

- **Data Flow Diagram:**

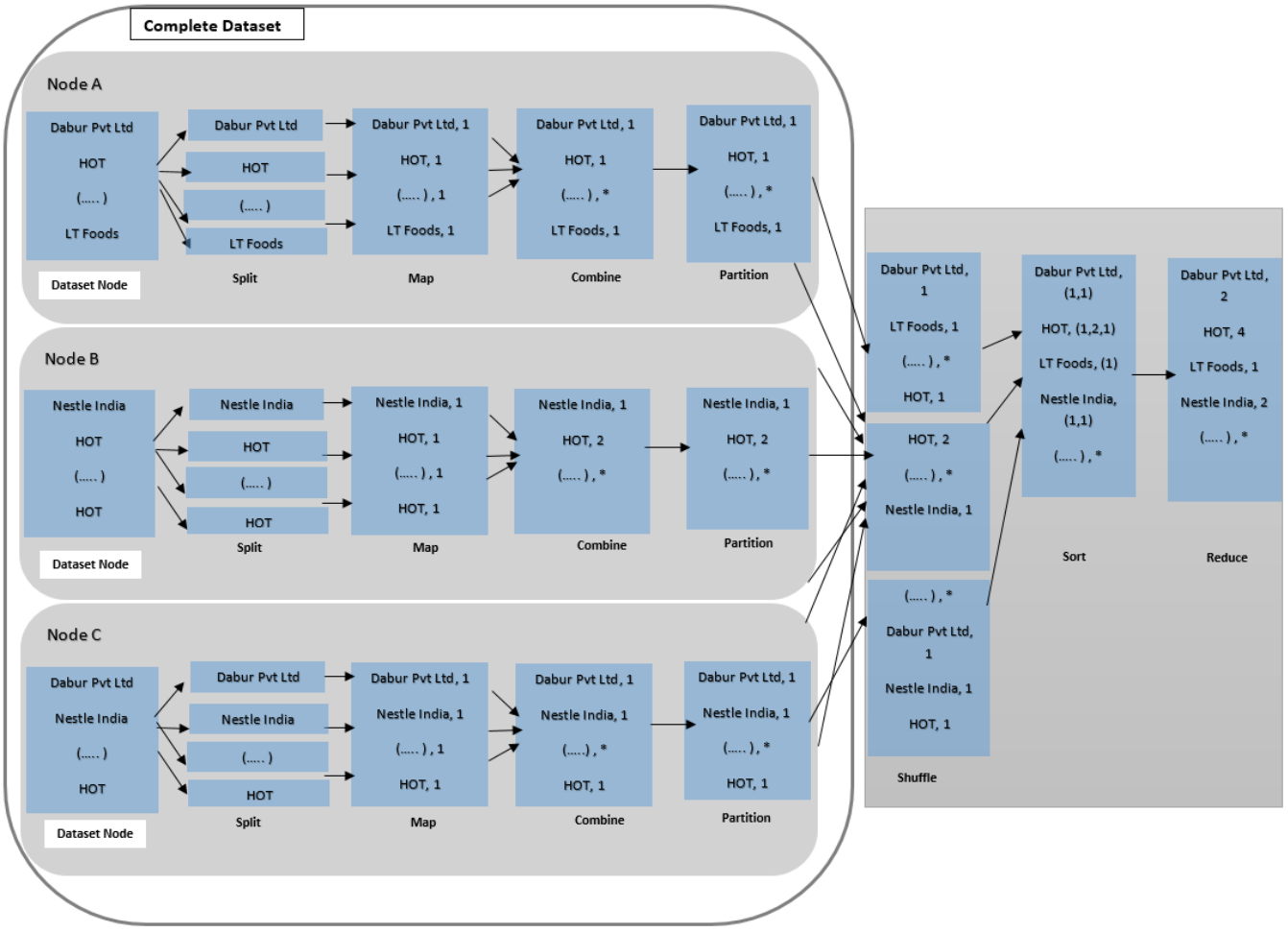


Fig 3: Hadoop Map-Reduce

By looking at this Hadoop map-reduce data flow diagram, it is seen that the complete dataset is being split into multiple nodes to store the database in chunks. Foe each of the nodes, the mapper and reducer operation will be processed. For this case scenario, the key which is manufacturer name will assign its occurrence in the mapping process, then in the combine part it will pre-reduce the occurrence of each manufacturer name. There will be no partitioning so by default the partitioning and reducing will be set to 1. After shuffling and sorting, the orders for each manufacturer will merge together and finally in the reduce phase the order counts will be aggregated under its manufacturer name.

- **Spark Jobs:**

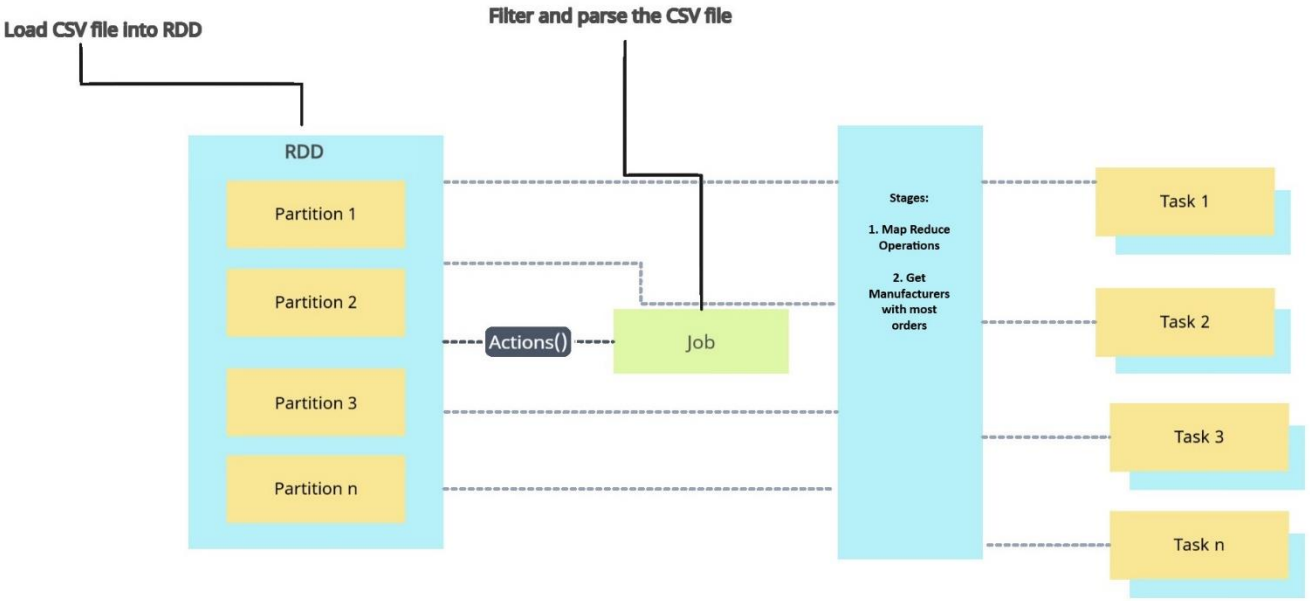


Fig 4: Spark Job Execution

First of all, in Spark, the CSV file is loaded into an RDD, which is a distributed collection of data across multiple partitions. Each partition can be processed independently. The RDD is divided into several partitions (Partition 1, Partition 2, ..., Partition n). Each partition contains a subset of the data. After the CSV file is loaded into the RDD, the data undergoes filtering and parsing operations. This step prepares the data for further processing. An action triggers the execution of the transformation steps defined on the RDD. Actions are operations that return a value to the driver program or write data to an external storage system. When an action is called, Spark creates a job to execute the transformations. The job consists of one or more stages, where each stage comprises a set of tasks.

The job is divided into stages based on shuffle boundaries. Stages represent a sequence of operations that can be executed together:

- MapReduce Operation: This stage typically involves a map operation followed by a reduce operation.

- Get Manufacturers with most orders: In this specific use case, the goal is to find the manufacturers with the most orders. This involves aggregating the data to count the number of orders per manufacturer.

Each stage is further divided into tasks, with one task per partition. These tasks represent the individual tasks that process each partition of the RDD. Each task operates on its respective partition and performs the required computations.

- **Result Discussion:**

Spark:

Firstly, I initialized a Spark session, which is the entry point to programming Spark with the DataFrame and Dataset API. I imported the final version of the CSV file into the Spark DataFrame and displayed the schema for a better view:

```
[2]: # Start of case studies
#Read the final dataset
# Create a SparkSession
spark = SparkSession.builder \
    .appName("CaseStudy1") \
    .getOrCreate()

# Read the CSV file into a DataFrame
case1 = spark.read.csv("C:/Big Data Project/dataset/cleaned_dataset/cleaned_dataset.csv", header=True, inferSchema=True)


# Show the updated DataFrame schema
case1.printSchema()
# Display the first few rows of the DataFrame
case1.show(10)

root
 |-- product_id: integer (nullable = true)
 |-- purchase_date: date (nullable = true)
 |-- city_name: string (nullable = true)
 |-- order_id: integer (nullable = true)
 |-- dim_customer_key: integer (nullable = true)
 |-- quantity: integer (nullable = true)
 |-- unit_selling_price: double (nullable = true)
 |-- total_discount_amount: double (nullable = true)
 |-- total_weighted_landing_price: double (nullable = true)
 |-- product_name: string (nullable = true)
 |-- product_type: string (nullable = true)
 |-- brand_name: string (nullable = true)
 |-- manufacturer_name: string (nullable = true)
 |-- product_category: string (nullable = true)
```

On the next step, I started grouping the DataFrame by the manufacturer_name column and counts the number of occurrences (orders) for each manufacturer. This renames the column from count to order_count for clarity. This sorts the manufacturers by their order counts in descending order, so the manufacturer with the most orders appears first. This displays the top 5 manufacturers with the highest order counts.

```
[12]: #Which manufacturer was the most popular in terms of getting orders than others? (Ranking based on items ordered)

# Group by 'manufacturer' and count 'order_id'
manufacturer_counts = case1.groupBy("manufacturer_name").count()

# Rename the 'count' column to 'order_count' for clarity
manufacturer_counts = manufacturer_counts.withColumnRenamed("count", "order_count")

# Order the results in descending order of 'order_count'
top_manufacturers = manufacturer_counts.orderBy(col("order_count").desc())

# Show the top 5 manufacturers
top_manufacturers.show(5)

[13]:
+--------------------+-----------+
|   manufacturer_name|order_count|
+--------------------+-----------+
|                 HOT|    2762333|
|               GCMMF|    2688754|
|Hindustan Unileve...|    1461404|
|         ITC Limited|    1379121|
|          Pepsi Food|    1179478|
+--------------------+-----------+
only showing top 5 rows
```

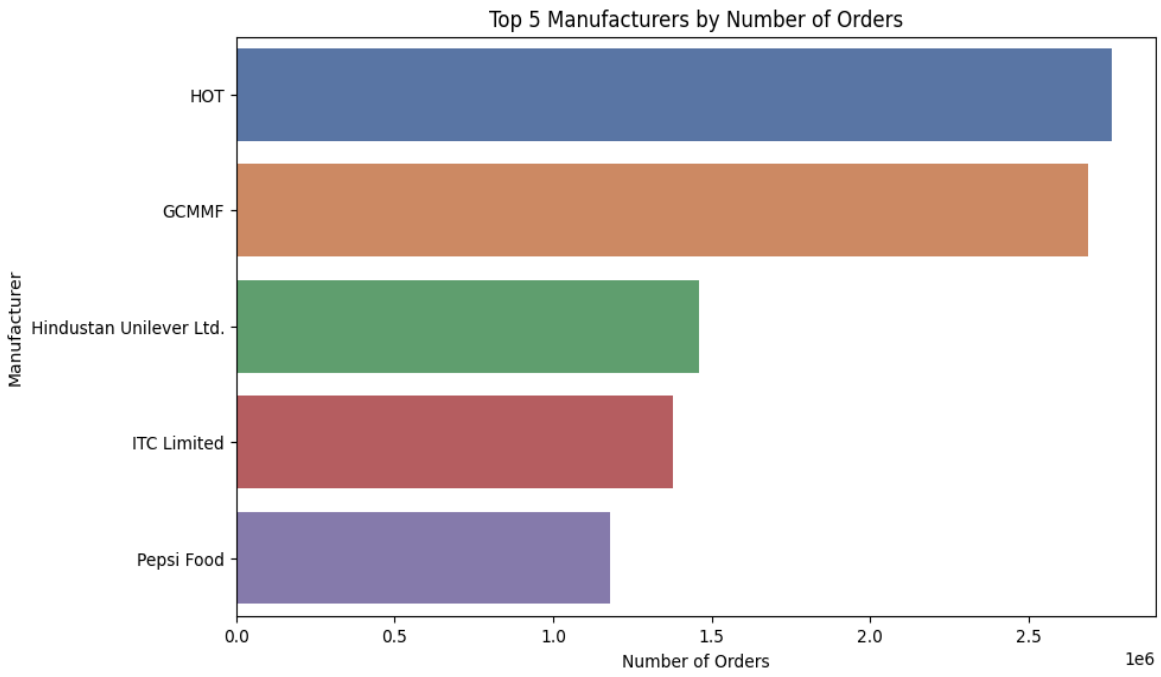


Fig 5: Bar Chart of top 5 manufacturers by items sold

Hadoop:

First of all, in Hadoop solution, I have stored the main and final dataset into the HDFS (Hadoop Distributed File System) before processing with all the operations and this will be the source of input file for the main dataset.

I have used python programming language and HDFS package to connect with my HDFS through my localhost for doing Hadoop operations. For mapping purpose, I am using the function to read the csv in clusters or chunks and yield the data rows as output and pass it to the reducer part.

```
# Connect to HDFS
client = InsecureClient('http://localhost:9870', user='hadoop')

# Path to your file in HDFS
file_path = '/final_project/cleaned_dataset.csv'

# Mapper

# Function to read chunks from HDFS
def read_hdfs_csv(file_path, chunksize=100000):
    with client.read(file_path) as reader:
        csv_stream = reader.__iter__()
        while True:
            chunk = []
            for _ in range(chunksize):
                try:
                    row = next(csv_stream).decode('utf-8').strip().split(',')
                    chunk.append(row)
                except StopIteration:
                    break
            if not chunk:
                break
            yield pd.DataFrame(chunk)
```

On the reducer part, it groups each chunk of rows yielded by the mapper by the 13th column (the manufacturer column), and counts the occurrences for each manufacturer. The counts from each chunk are concatenated to the agg_counts DataFrame which then sums the counts across all chunks for each manufacturer to get the total count of orders. Finally, getting the top 5 manufacturers by selecting the first 5 data from the descending order list.

```
# Reducer
# Initialize an empty DataFrame to store the aggregated counts
agg_counts = pd.DataFrame()

# Read the data in chunks
chunksize = 100000
data_chunks = read_hdfs_csv(file_path, chunksize=chunksize)

# Iterate over the chunks and concatenate the counts
for chunk in data_chunks:
    counts = chunk.groupby(12).size()  # Assuming 0-indexed columns, so 12 is the 13th column
    agg_counts = pd.concat([agg_counts, counts], axis=1, sort=False).fillna(0)

# Sum the counts for each manufacturer
agg_counts = agg_counts.sum(axis=1)

# Find the top 5 manufacturers
top_manufacturers = agg_counts.nlargest(5)
```

The final output is this:

```
hadoop@mashkur-1-2:~$ source /home/hadoop/.local/pipx/venvs/hdfs/bin/activate
(hdfs) hadoop@mashkur-1-2:~$ python3 /home/hadoop/Downloads/cleaned/case1.py
HOT                          2754346.0
GCMMF                        2688754.0
Hindustan Unilever Ltd.      1450240.0
ITC Limited                  1374594.0
Pepsi Food                   1179478.0
dtype: float64
(hdfs) hadoop@mashkur-1-2:~$
```

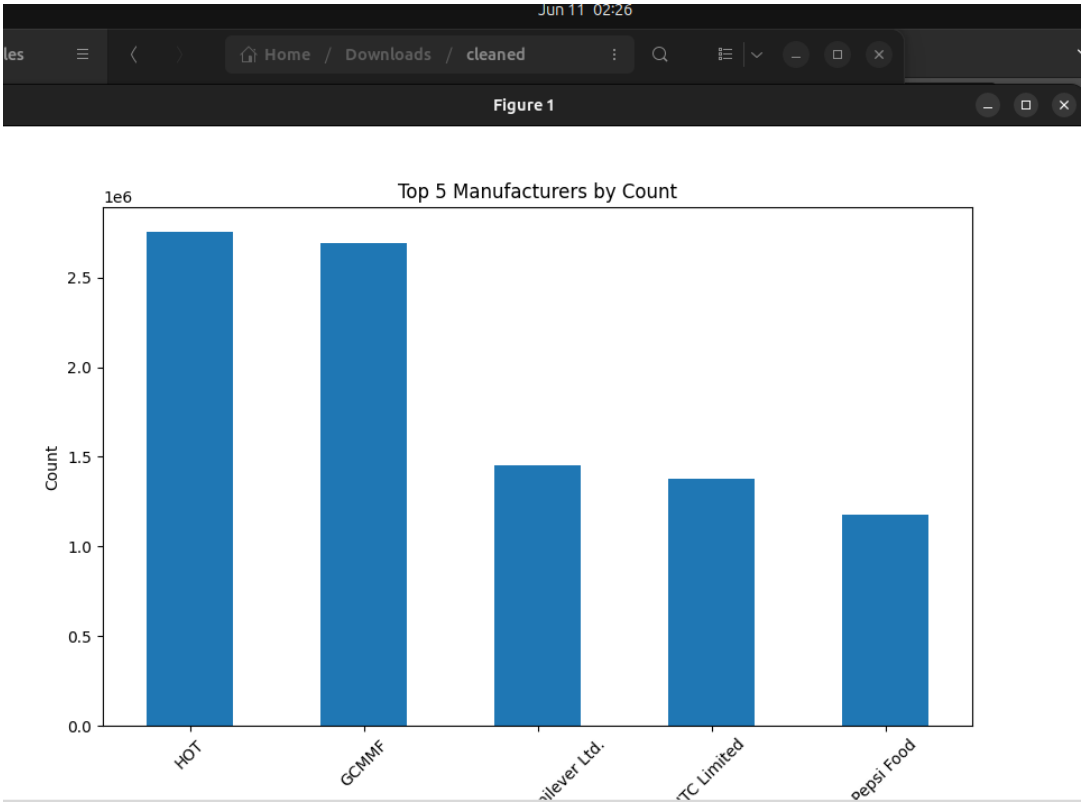


Fig 6: Bar Chart of top 5 manufacturers by items sold

From both the results, we can analyze that HOT is the most popular manufacturer in terms of sales whereas on the other hand Pepsi Food was the 5th most popular manufacturer on the list.

4.2 Case Study 02:

- **How many cities created orders more than 100,000 times?**

Key-Value Pair:

Key	Value
City	1 (each order is counted as 1)

In this MapReduce job, the mapping phase emits a pair for each order with the City as the key and a count of 1. The reduce phase then sums these counts to get the total number of orders for each city. Finally, the output filters and counts the cities where the total number of orders exceeds 100,000.

- **Data Flow Diagram:**

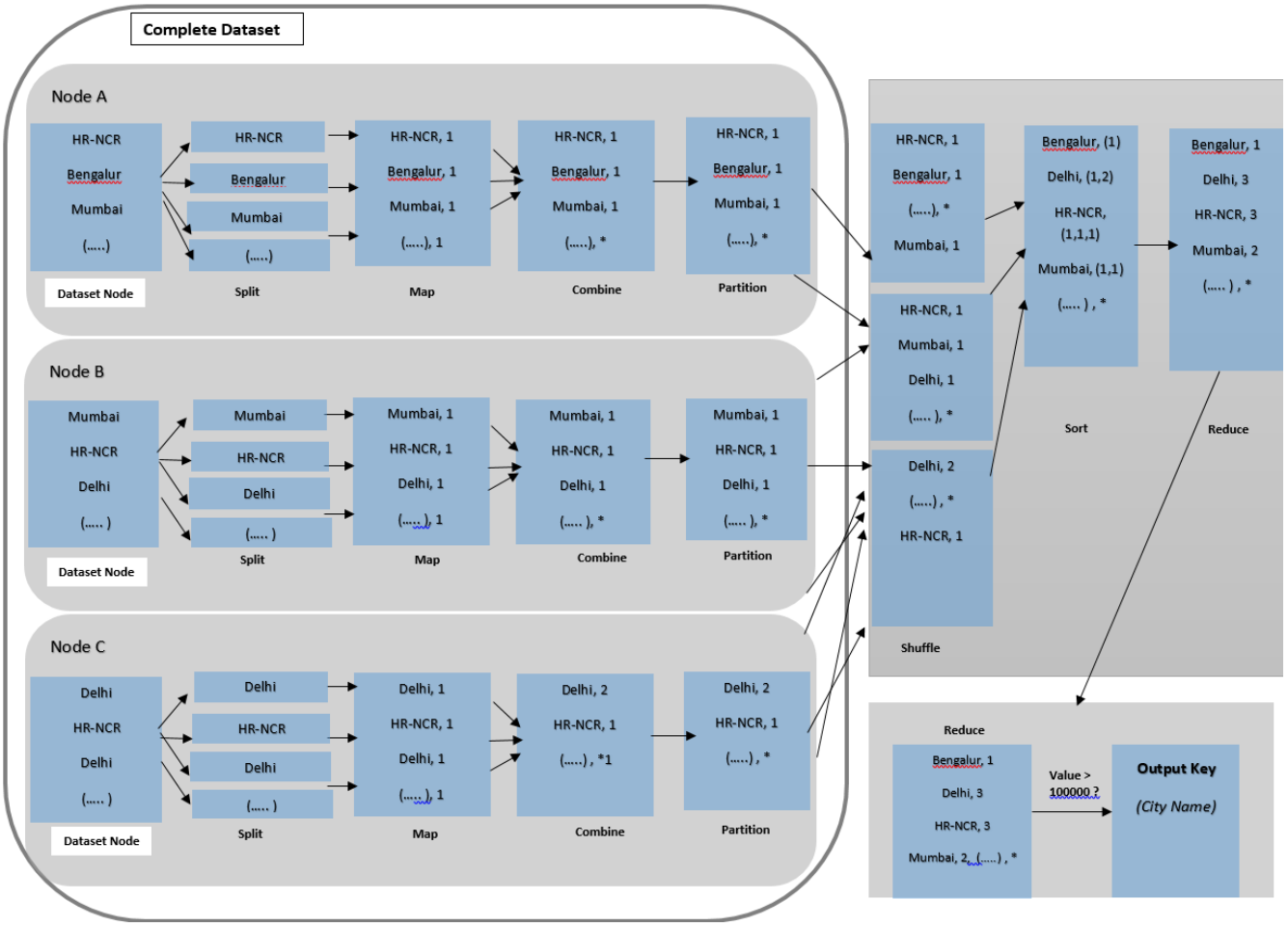


Fig 7: Hadoop Map-Reduce

By looking at this Hadoop map-reduce data flow diagram, it is seen that the complete dataset is being split into multiple nodes to store the database in chunks. For each of the nodes, the mapper and reducer operation will be processed. For this case scenario, the key which is the city name will assign its occurrence in the mapping process, then in the combine part it will pre-reduce the occurrence for each city name. There will be no partitioning so by default the partitioning and reducing will be set to 1. After

shuffling and sorting, the orders for each city name will merge together and finally in the reducer phase the order counts will be aggregated under its assigned city name. Ultimately, it will give us the total orders for each city but at last I will also apply a limit function to limit the orders more than 100000 times for each city resulting that the output key would be the city names with getting orders more than 100000 times individually. This will benefit us by getting insight to focus on cities of India that are mostly occupied with ordering items more compared to other cities on Flipkart e-commerce platform.

- **Spark Jobs:**

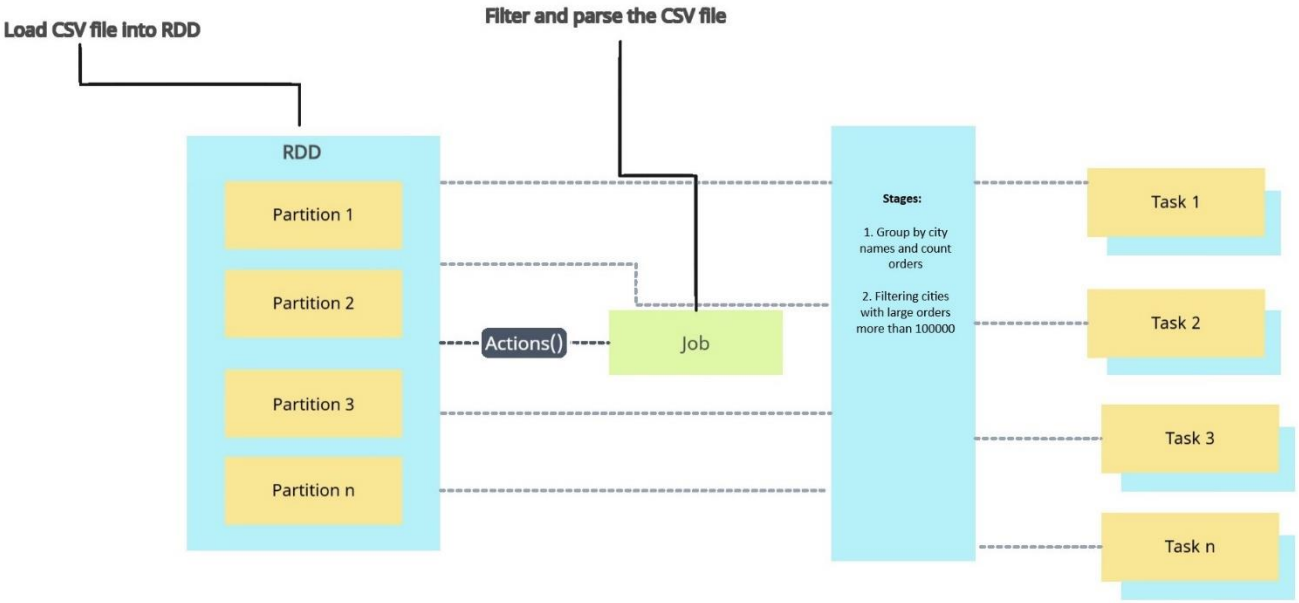


Fig 8: Spark Job Execution

First of all, in Spark, the CSV file is loaded into an RDD, which is a distributed collection of data across multiple partitions. Each partition can be processed independently. The RDD is divided into several partitions (Partition 1, Partition 2, ..., Partition n). Each partition contains a subset of the data. After the CSV file is loaded into the RDD, the data undergoes filtering and parsing operations. This step prepares the data for further processing. An action triggers the execution of the transformation steps defined on the RDD. When an action is called, Spark creates a job to execute the transformations. The job consists of one or more stages, where each stage comprises a set of tasks. In this case the job stages are:

- Grouping the data by city names and counting the number of orders for each city. This is typically done using a map-reduce operation where the data is first mapped to key-value pairs (city_name, 1) and then reduced by key to count the occurrences.
- In this stage, the results from the first stage are filtered to retain only those cities where the total number of orders exceeds 100,000. This involves applying a filter transformation to the RDD.

- **Result Discussion:**

Spark:

With the existing dataset read from the csv file, I proceeded to the logic execution which groups the DataFrame by the city_name column and counts the number of orders (or rows) for each city. The resulting DataFrame, city_order_counts, will have two columns: city_name and count. Then I filtered the city_order_counts DataFrame to retain only the rows where order_count is greater than or equal to 100,000.

The resulting DataFrame, cities_with_large_orders, contains only the cities that meet this criterion. Finally, I iterated over the collected list of Row objects and printed the name of each city that has more than 100,000 orders.

```
#Case study 2: How many cities created orders more than 100,000 times? (If I can't find any will reduce the cap/limit)
from wordcloud import WordCloud

# Group by 'city_name' and count 'order_id'
city_order_counts = case1.groupBy("city_name").count()

# Rename the 'count' column to 'order_count' for clarity
city_order_counts = city_order_counts.withColumnRenamed("count", "order_count")

# Filter cities with more than 100,000 orders
cities_with_large_orders = city_order_counts.filter(col("order_count") >= 100000)

# Collect the results
final_cities = cities_with_large_orders.collect()

# Print the city names
for city in final_cities:
    print(city["city_name"])

# Convert to Pandas DataFrame for visualization
cities_with_large_orders_pd = cities_with_large_orders.toPandas()
```

[28]:

	city_name	order_count
0	Mumbai	2892532
1	Delhi	16610829
2	Bengaluru	6263147
3	HR-NCR	8376708

Fig 9: Wordcloud of the most popular cities

From the above word cloud diagram, it is clearly seen that Delhi is the most popular city in India of getting orders more than 100000 times which puts it into the top of the list.

Hadoop:

In Hadoop solution, I have used python programming language and HDFS package to connect with my hdfs through my localhost for doing Hadoop operations. For mapping purpose, I am using the function to read the csv in clusters or chunks and yield the data rows as output and pass it to the reducer part same like the first solution.

```python
from hdfs import InsecureClient

# Connect to HDFS
client = InsecureClient('http://localhost:9870', user='hadoop')

# Path to your file in HDFS
file_path = '/final_project/cleaned_dataset.csv'

# Mapper
# Function to read chunks from HDFS
def read_hdfs_csv(file_path, chunksize=100000):
    with client.read(file_path) as reader:
        csv_stream = reader.__iter__()
        while True:
            chunk = []
            for _ in range(chunksize):
                try:
                    row = next(csv_stream).decode('utf-8').strip().split(',')
                    chunk.append(row)
                except StopIteration:
                    break
            if not chunk:
                break
            yield pd.DataFrame(chunk)
```

On the reducer part, it groups each chunk by the 3rd column (assuming 0-indexed columns, so column index 2 represents city_name), and counts the occurrences for each city. The counts from each chunk are concatenated to the agg_counts DataFrame which then sums the counts across all chunks for each manufacturer to get the total count of orders.

At the last part, I just filtered the cities to find those with more than 100,000 orders.

```python
# Reducer
# Iterate over the chunks and concatenate the counts
for chunk in data_chunks:
    counts = chunk.groupby(2).size()  # Assuming 0-indexed columns, so 2 is the 3rd column
    agg_counts = pd.concat([agg_counts, counts], axis=1, sort=False).fillna(0)

# Sum the counts for each city
agg_counts = agg_counts.sum(axis=1)

# Find the cities with more than 100,000 orders
popular_cities = agg_counts[agg_counts > 100000]
```

The final output is this:

```
(hdfs) hadoop@mashkur-1-2:~$ python3 /home/hadoop/Downloads/cleaned/case2.py
Bengaluru      6263147.0
Delhi         16610829.0
HR-NCR         8376708.0
Mumbai         2892532.0
dtype: float64
(hdfs) hadoop@mashkur-1-2:~$
```

It is seen that Delhi has a massive order count and way ahead of its competitor cities with orders of 16610829, this also clears the fact that it is the capital of India thus being the most popular city in terms of getting orders. We can also notice that although Mumbai is a very populated city in India but however it came 4th on this list with more than 2892532 orders. Not to mention Bengaluru and HR-NCR are also among these popular cities in terms of getting most orders more than 1 million times.

4.3 Case Study 03:

- **Which brands are offering the highest discount in Bengaluru, Mumbai and Delhi?**

Key-Value Pair:

Key	Value
City Name (i.e. "Bengaluru", "Mumbai", "Delhi")	The brand and the discount offered by that brand

This is to analyze the sales data to identify the brands that provide the greatest discounts in these specific cities. By focusing on these urban markets, the goal is to understand regional pricing strategies and consumer incentives. This information can be valuable for consumers looking for the best deals, and for businesses aiming to understand competitive pricing and market dynamics in major cities. In the context of the map-reduce operation—finding the brands offering the highest discount in Bengaluru, Mumbai, and Delhi—the key-value pairs would look something like this:

("Bengaluru", ("Brand1", 10))

("Mumbai", ("Brand2", 15))

("Delhi", ("Brand3", 20))

Each of these pairs represents a brand with its respective discount in a specific city. These key-value pairs would be processed using the map-reduce paradigm to find the brand with the highest discount in each city.

- **Data Flow Diagram:**

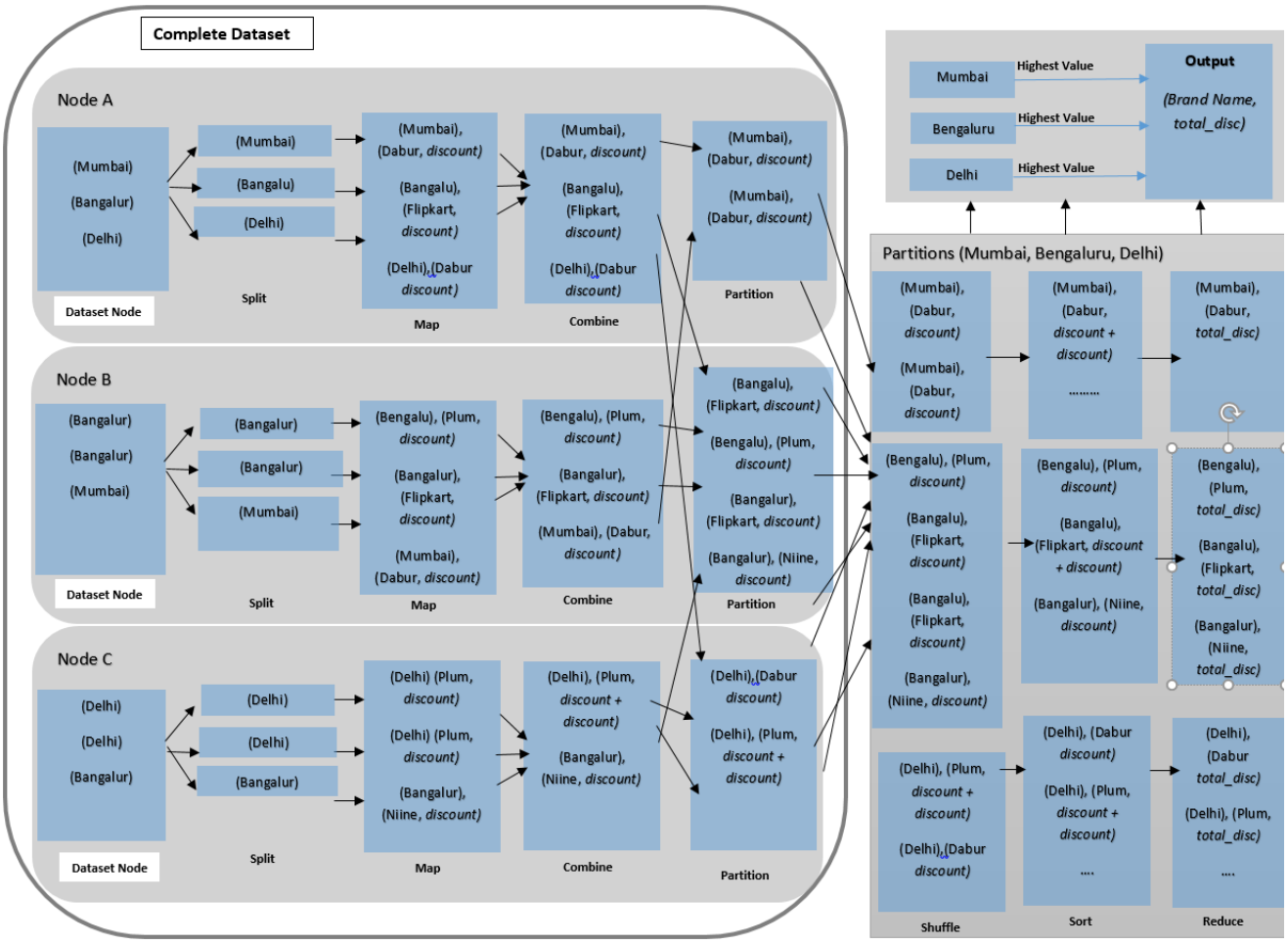


Fig 10: Hadoop Map-Reduce

By looking at this Hadoop map-reduce data flow diagram, it is seen that the complete dataset is being split into multiple nodes to store the database in chunks. For each of the nodes, the mapper and reducer operation will be processed. For this case scenario, the key which is the city name will be mapped based on the brand and corresponding discount value which will act as the overall value for this key. Then in the combine part it will pre-reduce the occurrence for each city name based on the sub-discounts for each brand under that city name. There will partitioning based on the 3 city names which are in this case Mumbai, Delhi and Bengaluru. For each partition the shuffle and sorting will produce the key-value pairs and merge all the discounts for each brand under each partition of cities. Finally, in the reducing phase the reducer will aggregate the total discount for the brands under each city and partition. And in the end, it will take the highest discount and its corresponding brand name for each city as value and the city name as its output key, showing the brands who are offering the highest discount in Bengaluru, Mumbai and Delhi including the discount value.

- **Spark Jobs:**

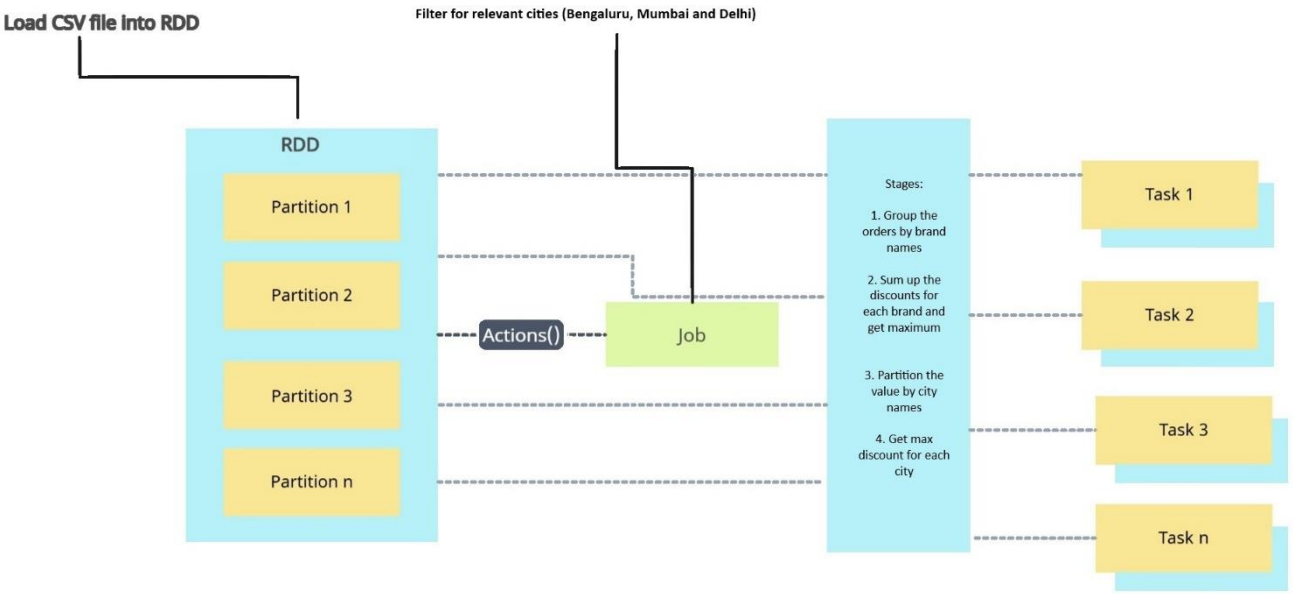


Fig 11: Spark Job Execution

First of all, in Spark, the CSV file is loaded into an RDD, which is a distributed collection of data across multiple partitions. Each partition can be processed independently. The RDD is divided into several partitions (Partition 1, Partition 2, ..., Partition n). Each partition contains a subset of the data. After loading the CSV file into the RDD, the data is filtered to include only the orders from the relevant cities: Bengaluru, Mumbai, and Delhi. This step narrows down the dataset to the cities of interest. An action triggers the execution of the transformation steps defined on the RDD. When an action is called, Spark creates a job to execute the transformations. The job consists of one or more stages, where each stage comprises a set of tasks.

The job is divided into stages based on shuffle boundaries. Stages represent a sequence of operations that can be executed together:

- This stage involves grouping the data by brand names. The data is mapped to key-value pairs (brand_name, order) and grouped by the brand names.
- Next, the discounts for each brand are summed up, and the brand offering the maximum discount is identified. This involves a reduce operation where the discounts are aggregated.
- The data is then partitioned by city names to ensure that each city's data is processed separately. This step is crucial for organizing the results by city.

- Finally, the maximum discount for each city is determined. This involves filtering and selecting the highest discount value for each city from the aggregated data.

- **Result Discussion:**

Spark:

With the existing dataset read from the csv file, I proceeded to the logic execution by filtering the DataFrame to include only the rows where the city_name column matches one of the cities in the list ["Bengaluru", "Mumbai", "Delhi"] and then repartitioned the DataFrame based on the city_name column to optimize the subsequent operations.

On the next step, I grouped the data by city_name and brand_name and calculates the maximum discount (total_discount_amount) for each brand in each city. The result is a DataFrame with columns: city_name, brand_name, and max_discount.

```
[16]: #Map-Reduce to Find Maximum Discount: Use the groupBy and agg functions to find the maximum discount by brand in each city.

from pyspark.sql import functions as F

df_max_discount = df_partitioned.groupBy("city_name", "brand_name")\
                                .agg(F.max("total_discount_amount").alias("max_discount"))
```

Now, after getting this dataframe I determined which brand offers the highest discount in each city by ranking the brands based on the maximum discount and filtering out the top-ranked brand to get the top brands for each city:

```
[19]: #Find the Top Brands for Each City: For each city, order the results by the maximum discount in descending order and get the top brand(s).
from pyspark.sql.window import Window
df_result = df_max_discount.withColumn("rank", F.rank().over(Window.partitionBy("city_name").orderBy(F.desc("max_discount"))))\
                           .filter("rank = 1")\
                           .drop("rank")

[23]: df_result.show(12, truncate=False)

+---------+---------------+------------+
|city_name|brand_name     |max_discount|
+---------+---------------+------------+
|Mumbai   |Amul           |1500.0      |
|Delhi    |India Gate     |1500.0      |
|Delhi    |Kellogg's      |1500.0      |
|Delhi    |Borges         |1500.0      |
|Delhi    |Gopaljee Ananda|1500.0      |
|Delhi    |Safe Harvest   |1500.0      |
|Bengaluru|Nandini        |1500.0      |
|Bengaluru|MamyPoko Pants |1500.0      |
|Bengaluru|GTS            |1500.0      |
|Bengaluru|Gold Flake     |1500.0      |
|Bengaluru|Marlboro       |1500.0      |
|Bengaluru|Enfamil        |1500.0      |
+---------+---------------+------------+
```

Hadoop:

In Hadoop solution, I have used python programming language and Hadoop Streaming for processing with the separate mapper.py and reducer.py files this time.

For mapping purpose, I am using the mapper.py file where I kept the mapping function. It reads the input CSV data and extracts relevant fields (city_name, brand_name, total_discount). Then I filtered

the data for Bengaluru, Mumbai, and Delhi and finally emitted the filtered data in the format: city_name, brand_name, total_discount.

```python
# Mapper function
def mapper():
    # Read each line from the standard input
    for line in sys.stdin:
        line = line.strip()
        # Parse the CSV line
        parts = list(csv.reader([line]))[0]

        # Extract relevant fields
        brand_name = parts[11]  # 12th column
        city_name = parts[2]    # 3rd column
        total_discount = parts[7]  # 8th column

        # Filter for cities of interest
        if city_name in ["Mumbai", "Bengaluru", "Delhi"]:
            # Emit brand name, city, and discount
            print(f"{city_name}\t{brand_name}\t{total_discount}")
```

On the other hand, there is a separate file created called reducer.py file consisting the reducer function which tracks the maximum discount and the brands offering that discount for each city and it finally outputs the city, brand, and maximum discount in the format: city_name, brand_name, max_discount.

```python
def reducer():
    current_city = None
    max_discount = 0
    max_discount_brands = set()

    for line in sys.stdin:
        line = line.strip()
        city_name, brand_name, discount = line.split('\t')
        discount = float(discount)

        if city_name != current_city:
            if current_city:
                # Output the results for the previous city
                for brand in max_discount_brands:
                    print(f"{current_city}\t{brand}\t{max_discount}")

            # Reset for the new city
            current_city = city_name
            max_discount = discount
            max_discount_brands = {brand_name}
        else:
            if discount > max_discount:
                max_discount = discount
                max_discount_brands = {brand_name}
            elif discount == max_discount:
                max_discount_brands.add(brand_name)

    # Output the results for the last city
    if current_city == city_name:
        for brand in max_discount_brands:
            print(f"{current_city}\t{brand}\t{max_discount}")
```

Partitioning: This execution code provided below is used to run the Hadoop Streaming job with specifying the mapper.py and reducer.py files separately to be running in the python kernal and also specifying the partition key field which is here set to 1 that indicates the first key which is in this case the city name as I will be partitioning based on the 3 city names and also it uses the built in partitioner

class. It also specifies the number of reducers to be 3, the same as the number of the partitioners which is based on the 3 different city names which are Mumbai, Bengaluru and Delhi.

```
hadoop@mashkur-1-2: ~
hadoop@mashkur-1-2:~$ hadoop jar /home/hadoop/Documents/hadoop-streaming-2.7.3.jar -file /hom
e/hadoop/Downloads/cleaned/case3_mapper.py -mapper "python3 /home/hadoop/Downloads/cleaned/ca
se3_mapper.py" -file /home/hadoop/Downloads/cleaned/case3_reducer.py -reducer "python3 /home/
hadoop/Downloads/cleaned/case3_reducer.py" -numReduceTasks 3 -partitioner org.apache.hadoop.m
apred.lib.KeyFieldBasedPartitioner -jobconf stream.num.map.output.key.fields=1 -jobconf num.k
ey.fields.for.partition=1 -input /final_project/cleaned_dataset.csv -output /final_project/ca
se3_demo
```

After running the job, the output is shown below. The output creates 3 partitions which stores the brand names offering the highest discount only in that particular city:

```
hadoop@mashkur-1-2: ~
hadoop@mashkur-1-2:~$ hdfs dfs -ls /final_project
Found 2 items
drwxr-xr-x   - hadoop supergroup          0 2024-06-11 12:25 /final_project/case3_demo
-rw-r--r--   1 hadoop supergroup 5266789998 2024-06-04 21:13 /final_project/cleaned_dataset.csv
hadoop@mashkur-1-2:~$ hdfs dfs -ls /final_project/case3_demo
Found 4 items
-rw-r--r--   1 hadoop supergroup          0 2024-06-11 12:25 /final_project/case3_demo/_SUCCESS
-rw-r--r--   1 hadoop supergroup        157 2024-06-11 12:25 /final_project/case3_demo/part-00000
-rw-r--r--   1 hadoop supergroup        122 2024-06-11 12:25 /final_project/case3_demo/part-00001
-rw-r--r--   1 hadoop supergroup         19 2024-06-11 12:25 /final_project/case3_demo/part-00002
hadoop@mashkur-1-2:~$ hdfs dfs -cat /final_project/case3_demo/part-00000
Bengaluru       Nandini 1500.0
Bengaluru       Marlboro        1500.0
Bengaluru       GTS     1500.0
Bengaluru       Enfamil 1500.0
Bengaluru       Gold Flake      1500.0
Bengaluru       MamyPoko Pants  1500.0
hadoop@mashkur-1-2:~$ hdfs dfs -cat /final_project/case3_demo/part-00001
Delhi   Safe Harvest    1500.0
Delhi   Kellogg's       1500.0
Delhi   India Gate      1500.0
Delhi   Gopaljee Ananda 1500.0
Delhi   Borges  1500.0
hadoop@mashkur-1-2:~$ hdfs dfs -cat /final_project/case3_demo/part-00002
Mumbai  Amul    1500.0
hadoop@mashkur-1-2:~$
```

The resulting data frame shows the final list of the brands in Mumbai, Delhi and Bengaluru which offer the best discount possible for that specific city. There are multiple brands offering the same highest discount based on this 3 cities however, in Mumbai there is only Amul brand which offers the highest discount up to 1500 rupees.

4.4 Case Study 04:

- **What are some of the target customers that spend the most money on Flipkart?**

Key-Value Pair:

Key	Value
Customer ID	Total spending by that customer on Flipkart

By calculating the total spending for each customer and then selecting the top 10 spenders, we can gain insights into the customer segments that contribute the most to Flipkart's revenue. Grouping the data by customer ID and summing the total spending for each customer, we create key-value pairs where the key is the customer ID and the value is the total spending. This allows us to easily identify the top spenders and visualize their contribution to Flipkart's revenue:

("Customer_ID_111", (total_spending_1))

("Customer_ID_221", (total_spending_2))

("Customer_ID_332", (total_spending_3))

...

Each key-value pair represents a customer ID (Customer_ID_111, Customer_ID_221, etc.) and their corresponding total spending (total_spending_1, total_spending_2, etc.). This structure allows us to easily access the total spending for each customer and analyze their contribution to Flipkart's revenue.

- **Data Flow Diagram:**

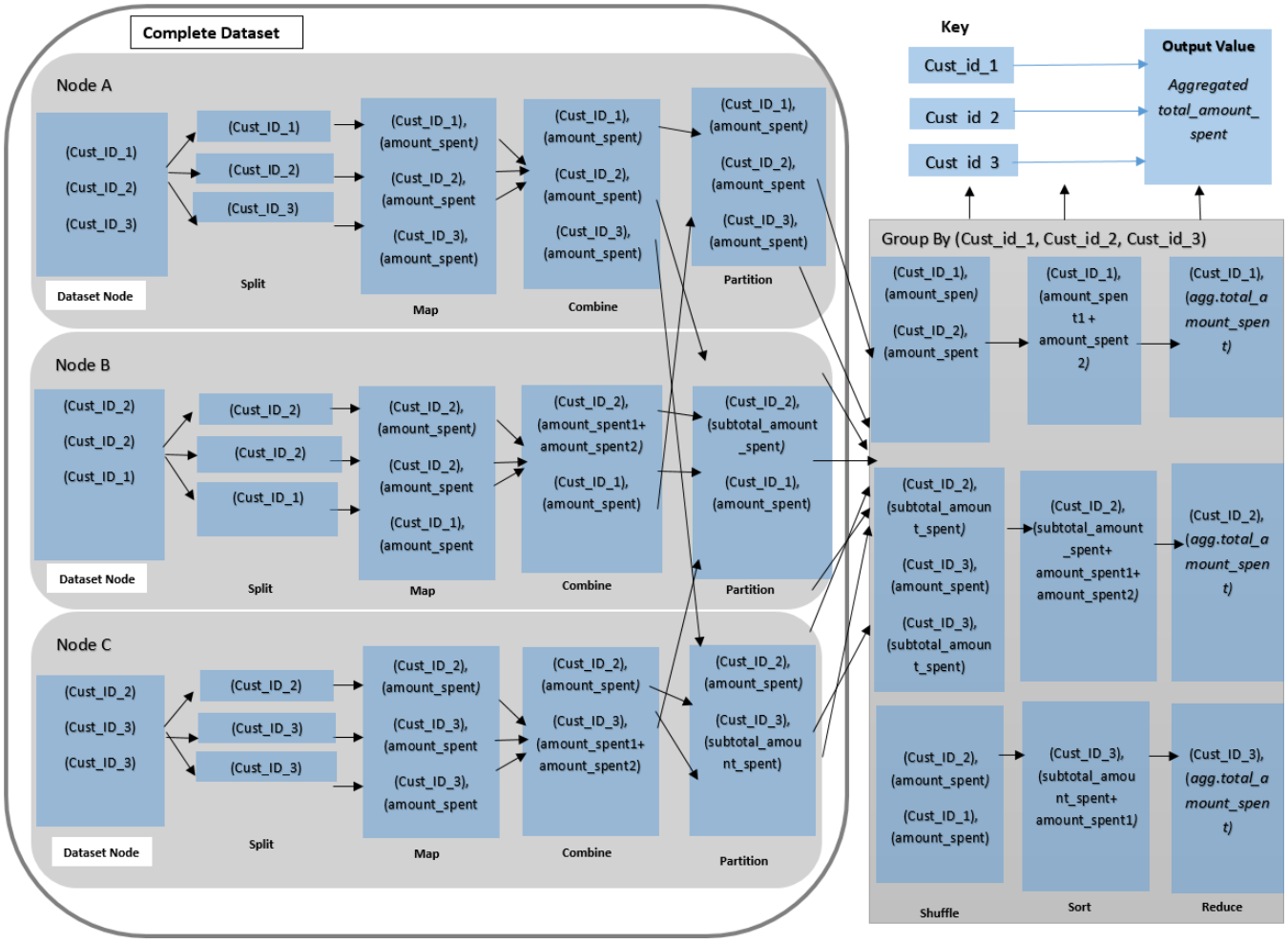


Fig 12: Hadoop Map-Reduce

By looking at this Hadoop map-reduce data flow diagram, it is seen that the complete dataset is being split into multiple nodes to store the database in chunks. For each of the nodes, the mapper and reducer operation will be processed. For this case scenario, the key which is the *dim_customer_key* also known as the customer_id will be mapped based on the total amount spent by that customer from the *total_weighted_landing_price* column. Then in the combine part it will pre-reduce the occurrence of the total spending of each customer and aggregate the sub total-spending. There will be no partitioning but it will group by the total spending for each customer. So the grouping of investments will be based on the customer IDs. In the shuffle and sorting phase, the key-value pairs will be randomly shuffled and sorted based on the key which is in this case the customer id. Finally, in the reducing phase it will aggregate all the sub total spending under each customer and produce output value as the total investment by customer IDs which will be the key in this case.

- **Spark Jobs:**

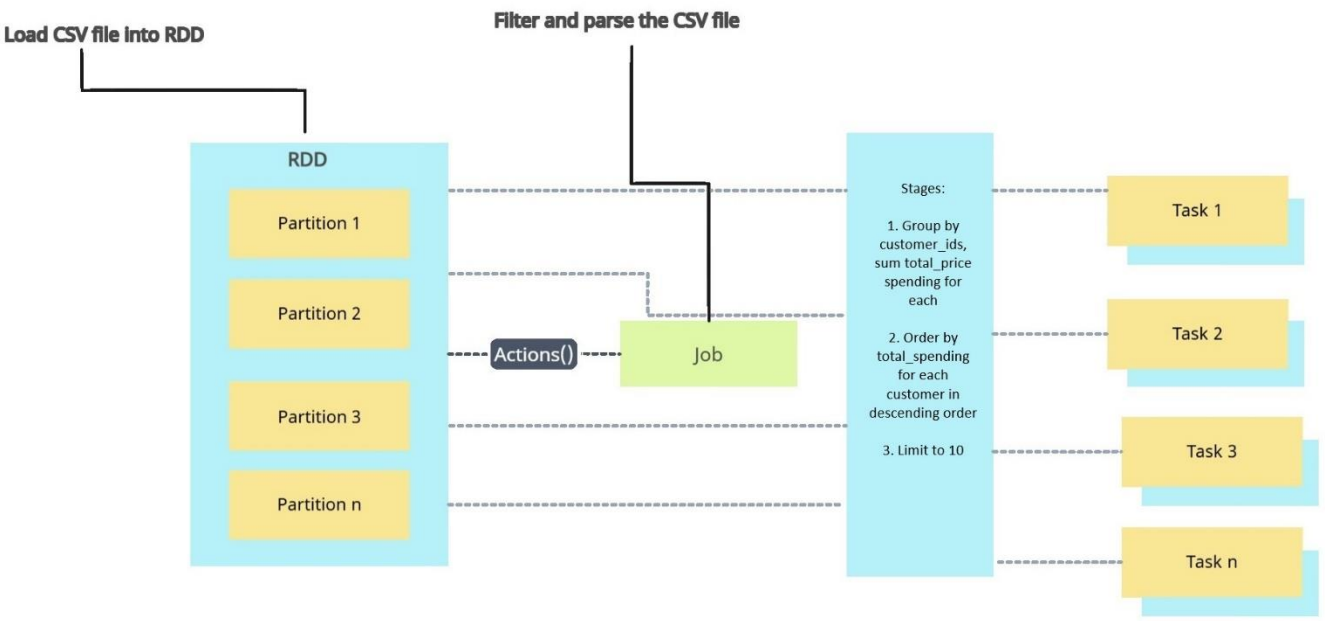


Fig 13: Spark Job Execution

First of all, in Spark, the CSV file is loaded into an RDD, which is a distributed collection of data across multiple partitions. Each partition can be processed independently. The RDD is divided into several partitions (Partition 1, Partition 2, ..., Partition n). Each partition contains a subset of the data. After loading the CSV file into the RDD, the data is parsed and is sent to the job for further execution. Then, the action represents operations that are performed on the RDD. When an action is called, Spark creates a job to execute the transformations. The job consists of one or more stages, where each stage comprises a set of tasks.

The job is broken down into stages, and each stage has multiple tasks. The tasks are the actual work units sent to the executor nodes which includes:

- Grouping the total spending by customer_id and then adding up total price.
- Ordering by the total spending for each customer in descending.
- Limit the output to the top 10 customers. This will show the target customers who spent most money on Flipkart e-commerce platform.

Each of the tasks within a stage performs operations on a partition of the RDD and the results are then combined.

- **Result Discussion:**

Spark:

After reading the csv file using spark, I grouped the DataFrame by the dim_customer_key column (representing customer IDs) and summed up the total_weighted_landing_price for each customer. The resulting DataFrame, customer_spending, has two columns: dim_customer_key and total_spending.

After that sorted the customer_spending DataFrame in descending order based on total_spending and selected the top 10 customers. The resulting DataFrame, top_10_customers, contains the 10 customers who spent the most.

```
# Load the CSV file into a DataFrame
case4 = spark.read.csv("C:/Big Data Project/dataset/cleaned_dataset/cleaned_dataset.csv", header=True, inferSchema=True)


# Group by customer_id and sum the total_landing_price for each customer
customer_spending = case4.groupBy("dim_customer_key").sum("total_weighted_landing_price")\
.withColumnRenamed("sum(total_weighted_landing_price)", "total_spending")

# Get the top 10 customers based on total spending
top_10_customers = customer_spending.orderBy(col("total_spending").desc()).limit(10)

# Show the top 10 customers
top_10_customers.show()
```

```
+----------------+--------------+
|dim_customer_key|total_spending|
+----------------+--------------+
|        12296465|      397001.0|
|        15147146|      185676.0|
|         4831341|      178704.0|
|         5271409|      166903.0|
|        12751874|      162886.0|
|         4414315|      160175.0|
|         1326909|      154209.0|
|        19568656|      145255.0|
|        14251312|      134121.0|
|        14794780|      120912.0|
+----------------+--------------+
```

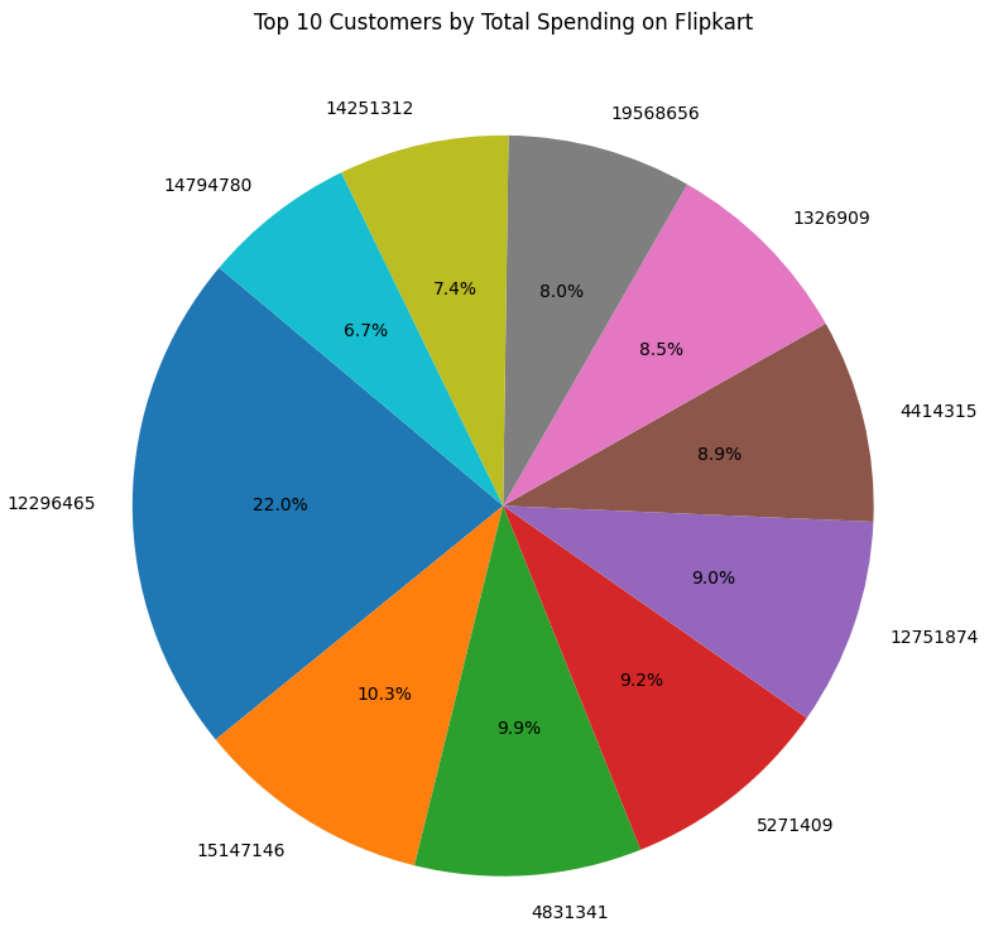


Fig 14: Pie Chart of the target customers

The resulting pie chart shows the target customers using their customer IDs to identify as the top investors in terms of shopping using this e-commerce platform.

Hadoop:

In Hadoop solution, I have used python programming language and Hadoop Streaming for processing with the separate mapper.py and reducer.py files this time.

For mapping purpose, I am using the mapper.py file where I kept the mapping function. It reads the input CSV data and extracts the customer_id from the 5th column and the total_landing_price from the 9th column.

```python
def mapper():
    reader = csv.reader(sys.stdin)
    for row in reader:
        if len(row) == 14:  # Ensure the row has all the columns
            try:
                customer_id = int(row[4])  # 5th column
                total_landing_price = float(row[8])  # 9th column
                print(f"{customer_id}\t{total_landing_price}")
            except (ValueError, IndexError):
                # Skip lines that don't convert correctly or where the indices are out of range
                continue
```

On the other hand, there is a separate file created called reducer.py file consisting the reducer function that uses a defaultdict to store the total spending for each customer. After processing all input from the mapper, the reducer splits the input data into customer_id and total_landing_price. It then adds the total_landing_price to the total spending for that customer and finally sorts the customers by total spending in descending order.

```python
def reducer():
    customer_spending = defaultdict(float)

    for line in sys.stdin:
        try:
            line = line.strip()
            customer_id, total_landing_price = line.split('\t')
            total_landing_price = float(total_landing_price)
            customer_spending[customer_id] += total_landing_price
        except ValueError:
            # Skip lines that don't split correctly or where the total_landing_price isn't a float
            continue

    # Sort customers by total spending in descending order and get the top 10
    top_customers = sorted(customer_spending.items(), key=lambda x: x[1], reverse=True)[:10]

    # Output the top 5 customers
    for customer_id, total_spending in top_customers:
        print(f"{customer_id}\t\t{total_spending}")
```

This execution code provided below is used to run the Hadoop Streaming job with specifying the mapper.py and reducer.py files separately to be running in the python kernel:

```
hadoop@mashkur-1-2: ~
hadoop@mashkur-1-2:~$ hadoop jar /home/hadoop/Documents/hadoop-streaming-2.7.3.jar -file /home/hado
op/Downloads/cleaned/case4_mapper.py -mapper "python3 /home/hadoop/Downloads/cleaned/case4_mapper.p
y" -file /home/hadoop/Downloads/cleaned/case4_reducer.py -reducer "python3 /home/hadoop/Downloads/c
leaned/case4_reducer.py" -input /final_project/cleaned_dataset.csv -output /final_project/case4_dem
o
```

After running the job, the output is shown below. These are the top target customers of Flipkart who spent the most for purchases:

```
hadoop@mashkur-1-2: ~
hadoop@mashkur-1-2:~$ hdfs dfs -ls /final_project
Found 3 items
drwxr-xr-x   - hadoop supergroup          0 2024-06-11 12:25 /final_project/case3_demo
drwxr-xr-x   - hadoop supergroup          0 2024-06-11 12:33 /final_project/case4_demo
-rw-r--r--   1 hadoop supergroup 5266789998 2024-06-04 21:13 /final_project/cleaned_dataset.csv
hadoop@mashkur-1-2:~$ hdfs dfs -ls /final_project/case4_demo
Found 2 items
-rw-r--r--   1 hadoop supergroup          0 2024-06-11 12:33 /final_project/case4_demo/_SUCCESS
-rw-r--r--   1 hadoop supergroup        186 2024-06-11 12:33 /final_project/case4_demo/part-00000
hadoop@mashkur-1-2:~$ hdfs dfs -cat /final_project/case4_demo/part-00000
12296465                397001.0
15147146                185676.0
4831341         178704.0
5271409         166628.0
12751874                162886.0
4414315         160175.0
1326909         154209.0
19568656                145255.0
14251312                134121.0
14794780                120912.0
hadoop@mashkur-1-2:~$
```

These are the top 10 customers who interacts the most with the company's revenue and income. It is also seen that the customer with ID 12296465 has spent the most with 397001 rupees, topping the chart with 22% of investment in total compared to others who are not even close to 20% of the investment.

5.0 Conclusion

Flipkart's sales data has been analyzed in great depth with Spark and Hadoop, which generated detailed understanding of consumer behavior, product behavior and trending sales. Secondly, by following well-established data pre-processing, cleaning, and validation practices, I were able to maintain the quality and internal consistency of the found results. The case studies have exposed the importance of manufacturer popularity, city-distribution of orders and discounting strategies of brands and buying behavior of consumers.

This project showcases what big data frameworks can do: churn out useful information from copious amounts of data. It helps us to have sophisticated tools like Spark and Hadoop paired with real-time analytics which are quintessential for a reseller based business to be operational in the competitive landscape of E-commerce.

In a nutshell, the understanding that is raised from this analysis can help Flipkart in optimizing all of its operations, personalization of marketing strategies and improving customer experience. The project demonstrates the importance of leveraging advanced data processing techniques to maintain a data-driven approach in the dynamic landscape of e-commerce.

References:

Chen, R., Mislove, A., & Wilson, C. (2015). Peeking beneath the hood of Uber. Proceedings of the 2015 ACM Conference on Internet Measurement Conference, 495-508.

Ghemawat, S., Gobioff, H., & Leung, S.-T. (2003). The Google file system. ACM SIGOPS Operating Systems Review, 37(5), 29-43.

Karau, H., & Warren, R. (2017). High Performance Spark: Best Practices for Scaling and Optimizing Apache Spark. O'Reilly Media.

Meng, X., Bradley, J., Yuvaz, B., Sparks, E., Venkataraman, S., Liu, D., ... & Zaharia, M. (2016). MLlib: Machine learning in Apache Spark. Journal of Machine Learning Research, 17(1), 1235-1241.

Thusoo, A., Sarma, J. S., Jain, N., Shao, Z., Chakka, P., Zhang, N., ... & Murthy, R. (2010). Hive: a petabyte scale data warehouse using Hadoop. Proceedings of the 2010 ACM SIGMOD International Conference on Management of Data, 117-128.

White, T. (2012). Hadoop: The Definitive Guide. O'Reilly Media.

Wu, X., Zhu, X., Wu, G.-Q., & Ding, W. (2014). Data mining with big data. IEEE Transactions on Knowledge and Data Engineering, 26(1), 97-107.

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). Spark: Cluster computing with working sets. Proceedings of the 2nd USENIX conference on Hot topics in cloud computing, 10-10.

Zaharia, M., Das, T., Li, H., Hunter, T., Shenker, S., & Stoica, I. (2013). Discretized streams: Fault-tolerant streaming computation at scale. Proceedings of the Twenty-Fourth ACM Symposium on Operating Systems Principles, 423-438.